UNIT 3 — CNN, Object Detection, Segmentation

What is a CNN?

A Convolutional Neural Network (CNN) is a neural network architecture designed to learn patterns from data with spatial structure. Images are the most common example.

A pixel is not independent of its neighbors. A small region may form an edge, corner, texture or object part. CNNs exploit this local structure using learnable filters.

```
Image
  ↓
Convolution
  ↓
Feature maps
  ↓
Activation (e.g., ReLU)
  ↓
Pooling / Downsampling
  ↓
More convolution blocks
  ↓
Flatten / Global pooling
  ↓
Classifier
  ↓
Class probabilities
```

Why Not Just Use a Fully Connected Network?

- A large image contains a very large number of pixels and therefore creates an enormous number of parameters in a fully connected layer.
- Fully connected layers do not naturally exploit the local spatial relationship between neighboring pixels.
- CNNs use local receptive fields and shared filter weights, which makes them much more parameter-efficient.
- The same filter can detect a useful pattern at different positions in the image.

Convolution Operation

A convolution layer applies a small matrix called a kernel or filter across an input. At each location, corresponding values are multiplied and summed.

```
Example input:
1 2 3
4 5 6
7 8 9

Example 2×2 filter:
1  0
0 -1

At the top-left:
(1×1) + (2×0) + (4×0) + (5×-1)
= -4
```

The filter slides over the image and produces a new matrix called a feature map.

What Does a CNN Filter Learn?

In classical computer vision, we often design filters manually. Examples include Sobel edge filters, Laplacian filters and Gaussian filters. In a CNN, filter values are learned from training data through backpropagation and optimization.

```
Early layers  → edges, simple textures
Middle layers → corners, motifs, object parts
Deep layers   → semantic structures and object-level patterns
```

Important distinction: The CNN is not explicitly told 'this filter is an edge detector'. Training discovers filter values that help minimize the task loss.

Kernel, Filter and Channel

For a color image, the input may have three channels: red, green and blue. A convolution filter spans all input channels.

```
Input: 32 × 32 × 3
Number of filters: 16
Kernel: 3 × 3

Output: 32 × 32 × 16   (with stride 1 and padding 1)
```

Each of the 16 filters produces one output feature map. Therefore, 16 filters create 16 output channels.

Padding

Padding adds values around the border of the input, commonly zeros. Without padding, repeated convolutions continuously shrink the spatial dimensions.

```
Input 32×32
3×3 convolution, stride 1, padding 0
Output = 30×30

Input 32×32
3×3 convolution, stride 1, padding 1
Output = 32×32
```

Why padding?: It preserves spatial dimensions when desired and gives border pixels more opportunity to influence learned representations.

Stride

Stride specifies how many pixels the kernel moves after each operation.

```
Stride 1 → moves one pixel at a time
Stride 2 → moves two pixels at a time
```

A larger stride usually reduces spatial resolution and computation.

Convolution Output Size

For one spatial dimension:

```
Output size = floor((N - F + 2P) / S) + 1
```

N = input size, F = filter size, P = padding, S = stride.

Example: N=32, F=3, P=1, S=1 gives 32.

```
floor((32 - 3 + 2×1)/1) + 1 = 32
```

Feature Maps

A feature map is the response produced by a filter as it scans the input. Strong activations indicate locations where the learned pattern is strongly present.

```
Input image
    ↓
Filter 1 → Feature map 1
Filter 2 → Feature map 2
Filter 3 → Feature map 3
...
Filter K → Feature map K
```

ReLU After Convolution

A common CNN block is convolution followed by a nonlinear activation such as ReLU.

```
ReLU(x) = max(0, x)
```

Without nonlinear activations, stacking linear operations would remain effectively linear and would severely limit what the network can learn.

Practical — Convolution with OpenCV

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)

# Vertical edge detector
kernel = np.array([
    [-1, 0, 1],
    [-1, 0, 1],
    [-1, 0, 1]
], dtype=np.float32)

edge = cv2.filter2D(image, -1, kernel)

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(image, cmap="gray")
plt.title("Original")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(edge, cmap="gray")
plt.title("Filter Response")
plt.axis("off")
plt.show()
```

Practical — PyTorch Conv2d

```
import torch
import torch.nn as nn

x = torch.randn(1, 3, 32, 32)

conv = nn.Conv2d(
    in_channels=3,
    out_channels=16,
    kernel_size=3,
    stride=1,
    padding=1
)

y = conv(x)

print("Input :", x.shape)
print("Output:", y.shape)

# Output:
# Input : torch.Size([1, 3, 32, 32])
# Output: torch.Size([1, 16, 32, 32])
```

POOLING OPERATIONS

Why Pooling?

Pooling reduces the spatial resolution of feature maps. It reduces computation and can make representations less sensitive to small translations.

```
32 × 32 × 64
      ↓
2×2 pooling, stride 2
      ↓
16 × 16 × 64
```

Key point: Pooling changes height and width, but normally does not change the number of channels.

Max Pooling

```
Input:
1  3  2  4
5  6  1  2
7  2  8  3
4  1  2  9

2×2 max pooling, stride 2:

6  4
7  9
```

Max pooling keeps the strongest activation in each local region.

Average Pooling

Average pooling computes the mean of values inside each pooling window.

```
For:
1 3
5 6

Average = (1+3+5+6)/4 = 3.75
```

Max vs Average Pooling

Property	Max Pooling	Average Pooling
Operation	Maximum	Mean
Typical effect	Keeps strongest response	Smooths local response
Common use	Feature downsampling	Downsampling / global average pooling
Intuition	Is the feature strongly present?	What is the average activation?

Practical — Pooling in PyTorch

```
import torch
import torch.nn as nn

x = torch.randn(1, 3, 32, 32)

max_pool = nn.MaxPool2d(kernel_size=2, stride=2)
avg_pool = nn.AvgPool2d(kernel_size=2, stride=2)

print(max_pool(x).shape)
print(avg_pool(x).shape)

# Both:
# torch.Size([1, 3, 16, 16])
```

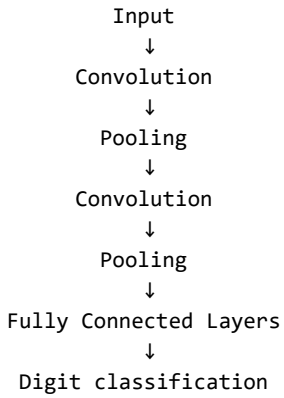
CLASSICAL CNN ARCHITECTURES

LeNet-5

LeNet-5 (1998) was designed by Yann LeCun for handwritten digit recognition, especially MNIST-like images.

Architecture:

Input 32×32×1
→ **C1: Conv 5×5, 6 filters**
→ **S2: Average Pooling 2×2**
→ **C3: Conv 5×5, 16 filters**
→ **S4: Average Pooling 2×2**
→ **C5: Conv 5×5, 120 filters**
→ **F6: Fully Connected 84**
→ **Output: 10 classes**



Its importance is conceptual: it demonstrated that convolutional feature extraction followed by learned classification can work extremely well for visual recognition tasks.

LeNet-Style Practical Model

```
import torch
import torch.nn as nn

class LeNet(nn.Module):
    def __init__(self):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv2d(1, 6, 5),
            nn.ReLU(),
            nn.AvgPool2d(2),

            nn.Conv2d(6, 16, 5),
            nn.ReLU(),
            nn.AvgPool2d(2)
        )

        self.classifier = nn.Sequential(
```

```
            nn.Flatten(),
            nn.Linear(16 * 4 * 4, 120),
            nn.ReLU(),
            nn.Linear(120, 84),
            nn.ReLU(),
            nn.Linear(84, 10)
        )

    def forward(self, x):
        return self.classifier(self.features(x))

model = LeNet()
print(model)
```

AlexNet

AlexNet was a landmark CNN that demonstrated the power of deep learning at large scale for image classification.

- Used ReLU activations extensively.
- Made effective use of GPU computation.
- Used dropout for regularization.
- Used data augmentation.
- Contained substantially more learned capacity than earlier CNNs.

Historical significance: AlexNet helped establish deep CNNs as a dominant approach to large-scale visual recognition.

Architecture:

Input 227×227×3
→ **Conv1: 11×11, 96**
→ **Max Pool**
→ **Conv2: 5×5, 256**
→ **Max Pool**
→ **Conv3: 3×3, 384**
→ **Conv4: 3×3, 384**
→ **Conv5: 3×3, 256**
→ **Max Pool**
→ **FC 4096**
→ **FC 4096**
→ **Output 1000**

Practical — Loading AlexNet

```
import torch.nn as nn
from torchvision.models import alexnet, AlexNet_Weights

model = alexnet(weights=AlexNet_Weights.DEFAULT)

# Example: adapt to 10 classes
model.classifier[6] = nn.Linear(
    model.classifier[6].in_features,
    10
)

print(model)
```

ADVANCED CNN ARCHITECTURES

VGG-16

VGG-16 is a deep convolutional neural network (CNN) architecture introduced by Karen Simonyan and Andrew Zisserman from the Visual Geometry Group (VGG) at the University of Oxford in 2014.

Input Layer: 224 × 224 × 3 (RGB image)

1. **Block 1: Two Conv2D layers (64 filters, 3 × 3), followed by MaxPool (2 × 2)**
2. **Block 2: Two Conv2D layers (128 filters, 3 × 3), followed by MaxPool (2 × 2)**
3. **Block 3: Three Conv2D layers (256 filters, 3 × 3), followed by MaxPool (2 × 2)**
4. **Block 4: Three Conv2D layers (512 filters, 3 × 3), followed by MaxPool (2 × 2)**
5. **Block 5: Three Conv2D layers (512 filters, 3 × 3), followed by MaxPool (2 × 2)**
6. **Fully Connected Layer 1 (Dense): 4,096 channels**
7. **Fully Connected Layer 2 (Dense): 4,096 channels**
8. **Output Layer (Dense/Softmax): 1,000 channels (for ImageNet classification)**

Main idea: Use repeated small convolutions to build a deep hierarchical representation.

Limitation: VGG is computationally and parameter intensive, especially compared with later efficient designs.

Challenges in Deep Neural Networks

Deep Neural Networks are powerful models, but training them becomes difficult as network depth increases. Two major issues are:

1. Vanishing/Exploding Gradient Problem: As the number of layers increases, gradients can become extremely small (vanishing) or very large (exploding) during backpropagation, making training unstable.

2. Degradation Problem: Increasing network depth does not always improve performance and can even degrade it.

ResNet

- A **residual neural network** (also referred to as a **residual network** or **ResNet**) is a deep learning architecture in which the layers learn residual functions with reference to the layer inputs.
- It was developed in 2015 for image recognition, and won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) of that year.
- ResNet introduced residual or skip/residual connections to make very deep networks easier to optimize minimizing the degradation problem.



In a typical ResNet basic block, there are two convolutional layers, Batch Normalization, activation functions, and the shortcut connection.

Why it helps: Skip connections which makes it possible to train very deep neural networks by allowing information and gradients to bypass some layers.

Practical — ResNet

```
from torchvision.models import resnet18, ResNet18_Weights
import torch.nn as nn

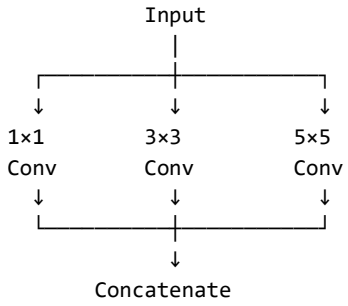
model = resnet18(weights=ResNet18_Weights.DEFAULT)

model.fc = nn.Linear(
    model.fc.in_features,
    10
)

print(model)
```

Inception Networks (GoogLeNet)

The Inception idea is to process information at multiple spatial scales inside a block.



This allows the network to learn patterns at different scales rather than committing to a single kernel size.

Useful when want to learn diverse image features efficiently.

MobileNet

A conventional CNN that uses standard convolution layers perform two jobs simultaneously:

- Extract spatial features, such as edges, textures, and shapes.
- Combine information from different input channels.

Doing both jobs together can be computationally expensive, particularly on mobile phones and embedded devices.

MobileNet addresses this using a technique called depthwise separable convolution.

```
Standard convolution:
Input → full convolution → Output


MobileNet style Architecture (Depthwise separable):
Input
 ↓
Depthwise convolution
 ↓
Pointwise 1×1 convolution
 ↓
Output
```

Depthwise convolution processes spatial information channel-by-channel, i.e each input channel gets its own spatial filter.

For example:
Input channels = 32
Filters per input channel = 1
Filter size = 3 * 3

So each of the 32 channels is processed independently. It does not combine information across channels. It only extracts spatial features within each channel.

Now we need to combine the information from 32 channels and generate 64 output channels if we have 64 filters in convolution layer for MobileNet uses a 1×1 convolution, called pointwise convolution which mixes information across channels.

Advantages of using MobileNet:

Suppose input feature map diemsion : 112 * 112 * 32 and we want to produce 64 output channels using a 3*3 convolution.

In standard convolution, each filter has dimensions: 3*3*32 and 64 filters are there

So total weights = $3 \times 3 \times 32 \times 64 = 18{,}432$

For depthwise 3 * 3 is size of each filter and we have 32 channels

The number of weights is: $3 \times 3 \times 32 = 288$

And now 1 by 1 pointwise convolution will be used across all 32 channels to combine information

And we want 64 channels output → $1 \times 1 \times 32 \times 64 = 2{,}048$ weights

Practical — MobileNet

```
from torchvision.models import mobilenet_v2, MobileNet_V2_Weights
import torch.nn as nn

model = mobilenet_v2(weights=MobileNet_V2_Weights.DEFAULT)

model.classifier[1] = nn.Linear(
    model.classifier[1].in_features,
    10
)

print(model)
```

Architecture Comparison

Architecture	Core idea	Problem / goal
LeNet-5	Early convolution + pooling pipeline	Learn visual features for recognition
AlexNet	Deep CNN + ReLU + GPU + dropout	Large-scale image classification
VGG-16	Stacks of 3×3 convolutions	Simple, deep architecture
ResNet	Residual/skip connections	Train very deep networks
Inception	Parallel multi-scale branches	Capture features at multiple scales
MobileNet	Depthwise separable convolution	Reduce computation for efficient deployment